Université Grenoble Alpes
L3 Informatique

Année 2026-2027

Architectures logicielles et matérielles
Semaine 5 TD 1

Questions à traiter en début de séance. (remise à 8h15)

Question 1.
Rappeler les rôles des 3 principales phases de traitement qu'un processeur doit réaliser **pour chaque instruction** d'un programme en exécution :

- quel est le rôle de la phase de "**fetch**" ? quel registre du processeur met-elle à jour pour utilisation par la phase suivante ?

- quel est le rôle de la phase de "**decode**" ? sur le contenu de quel registre travaille-t-elle ? est-elle réalisée par la partie contrôle ou la partie opérative du processeur ?

- quel est le rôle de la phase de "**execute**" ? est-elle réalisée par la partie contrôle ou la partie opérative du processeur ? grâce à quels éléments matériels cette partie du processeur est-elle pilotée (i.e. "sait-elle" ce qu'elle doit effectuer, par ex une addition et pas une soustraction) ?

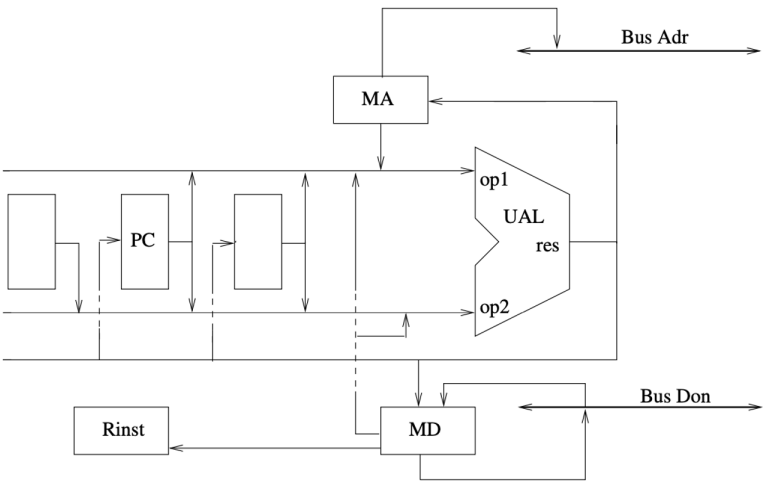
Question 2.
Quelle est conceptuellement la forme de la **partie contrôle** d'un processeur ?
Dans cette représentation, qu'est-ce qui est commun à toutes les instructions ?

Question 3.
3.1 Lors de l'exécution d'un programme, **à quels moments** (dans les phases de traitement rappelées en question 1) la présence d'une **interruption** peut-elle être prise en compte par le processeur ?
3.2 De façon générale, quelles sont les **actions** (sauvegardes, mises à jour) alors réalisées pour **traiter le départ** en interruption, et pourquoi ?

Question 4.
De façon générale, quelles sont les actions (sauvegardes, mises à jour) réalisées au **retour** d'interruption ?

Exercice.

Nous illustrons ici **l'interprétation** des instructions, et le traitement des **interruptions**, sur un exemple de processeur simple fictif, mais plus proche d'un processeur (ARM) réaliste que l'exemple jouet vu en cours.
La *partie opérative* de ce processeur se présente comme rappelé au cours 2, avec notamment la présence des **registres** PC (program counter), Rinst (ou IR, registre instruction), MA (memory address register), et MD (memory data register) :



Ce processeur possède *16 registres généraux* (R0 à R15) et *3 registres spéciaux*.
Parmi les registres généraux, 2 registres ont une fonction particulière (comme dans un processeur ARM) :

- R15 qui joue le rôle de PC
- et R14 ou LR (link register, sauvegarde de l'adresse de retour lors des appels de fonctions)

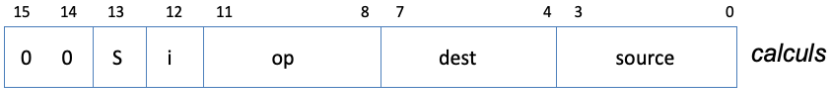
Les 3 registres spéciaux sont :

- **cpsr** (current processor status register) : registre état courant. Nous ne mentionnerons ici que ses bits ZNCV stockant les indicateurs produits par l'UAL (et utilisés par les instructions de branchement) et un bit I associé à la gestion des interruptions.
- **spsr** (save processor status register), pour la sauvegarde de cpsr
- **slr**, pour la sauvegarde de l'adresse de retour en cas d'interruption

Les **instructions** sont toutes codées sur 16 bits (elles doivent donc être stockées en mémoire à des adresses paires, et PC est incrémenté de 2 octets après chaque instruction). Elles sont regroupées en 4 catégories (chacune étant dotée d'un format binaire) :

- calcul
- transfert mémoire (load/store)
- branchement
- instruction spéciale

- 1 - Instructions de calcul : *<Operation> Rdest, Rsource* ou *<Operation>S Rdest, Rsource*



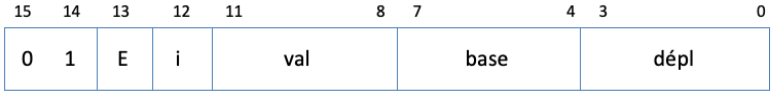
calculs

Le premier opérande dénote aussi la destination du résultat du calcul (tous deux sont stockés dans le même registre).
Le champ *op* dans le format binaire indique la nature du calcul réalisé, par exemple :

SUB (0000) : soustraction Dest ← Dest - Source
ADC (0001) : addition avec retenue
RSB (0010) : reverse sub Dest ← Source - Dest
RSC (0011) : reverse sub avec retenue
...

Selon le bit i (numéro 12), l'opérande2 est peut-être soit une constante entière #cte, soit le contenu d'un registre source. Le bit S (numéro 13) indique si les bits de cpsr stockant les indicateurs ZNCV doivent être mis à jour.

- 2 - Instructions de transfert mémoire : ***LDR*** *Rval, [Rbase, depl]* ou ***STR*** *Rval, [Rbase, depl]*

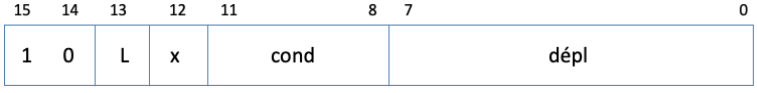


load / store

L'adresse mémoire utilisée (spécifiée entre crochets) est la somme du contenu du registre Rbase et d'un déplacement. Selon le bit i (numéro 12), le déplacement est soit une constante entière #depl, soit le contenu du registre dépl.
Le bit E (numéro 13) indique s'il s'agit d'une instruction LDR ou STR :

ldr, c'est à dire Rval ← Mem[Rbase + depl], si E = 0
str, c'est à dire Rval → Mem[Rbase + depl], si E = 1.

- 3 - Instructions de branchement : ***B****{<cond>} label* ou ***BL****{<cond>} label*



bcond / blcond

Le déplacement, relatif au compteur ordinal (*program counter*), est codé sur un octet.
Le bit L (numéro 13) indique un saut ordinaire (si L = 0) ou bien un saut avec sauvegarde de PC dans le registre LR (si L = 1).
Les conditions testables sont identiques à celles offertes par un processeur ARM.

- 4 - Instructions spéciales :
Nous ne détaillerons pas les instructions disponibles dans cette catégorie, elles permettent de faire des transferts entre registres. Notons seulement leur codage :

15	14	13	12	11	10	9	8	7 … 4	3 … 0	
1	1	0	0	1	1	0	1	Rspec	Rgen	*Rspec → Rgen*
1	1	0	1	1	1	0	1	Rgen	Rspec	*Rgen → Rspec*

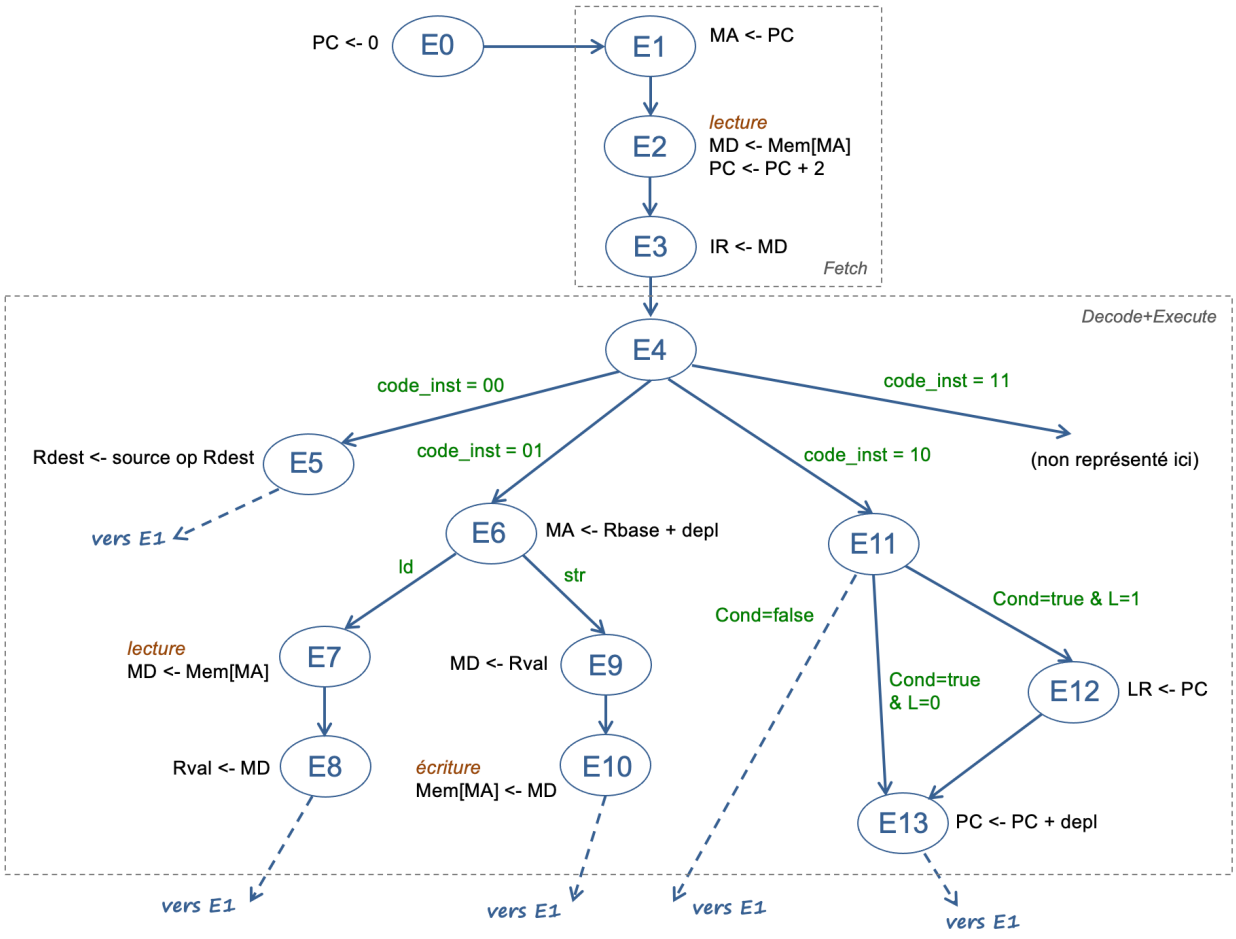
==========

Q1. Pour chacune des instructions suivantes, indiquer :
- à quelle catégorie elle appartient
- ce qu'elle réalise

```
1) sub r2, r11
2) ldr r12, [r5, #4]
3) bne 32
```

Q2. La figure ci-dessous donne une vue simplifiée de *l'automate d'interprétation* pour les instructions de ce processeur (par ex, seul le cas d'un déplacement constante entière est pris en compte pour les transferts mémoire).

Reconnaître dans cette figure les phases de traitement appliquées à chaque instruction, et **expliquer** l'interprétation des instructions de type *calcul*, *transfert mémoire*, et *branchement*, ainsi que la raison du retour à l'état E1 en fin de traitement (schématisé en pointillés pour alléger le dessin).



Q3. Dans **quels états** de cet automate d'interprétation est-il nécessaire de tester si une requête d'interruption est à prendre en compte (donner leurs numéros) et pourquoi ?

Q4. Quelle **condition** devra être satisfaite pour que l'exécution continue par un traitement d'interruption ?

Q5. En conséquence, dessiner les **modifications de l'automate** nécessaires pour gérer les requêtes d'interruptions externes.

Q6. Enfin, dessiner également dans l'automate le traitement de **l'instruction rti** pour le retour d'interruption. Le codage de cette instruction est le suivant :

1	1	1	1	x	x	x	x	x	x	x	x	x	x	x	x

rti